

Course Code: DCIT 208

Assignment Number: Assignment 2

Assignment Title: Request Flow Hand Drawing Submission

Full Name: KWASHI ELORM FREMA

Student ID: 22397307

Team Name: SEG26-33-MerakiArtsAfrica

Project Title: Meraki Arts

Team Repository URL: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-33-merakiartsafrica>

Individual Portfolio Repository URL: <https://github.com/Elorm07/dcit208-portfolio.git>

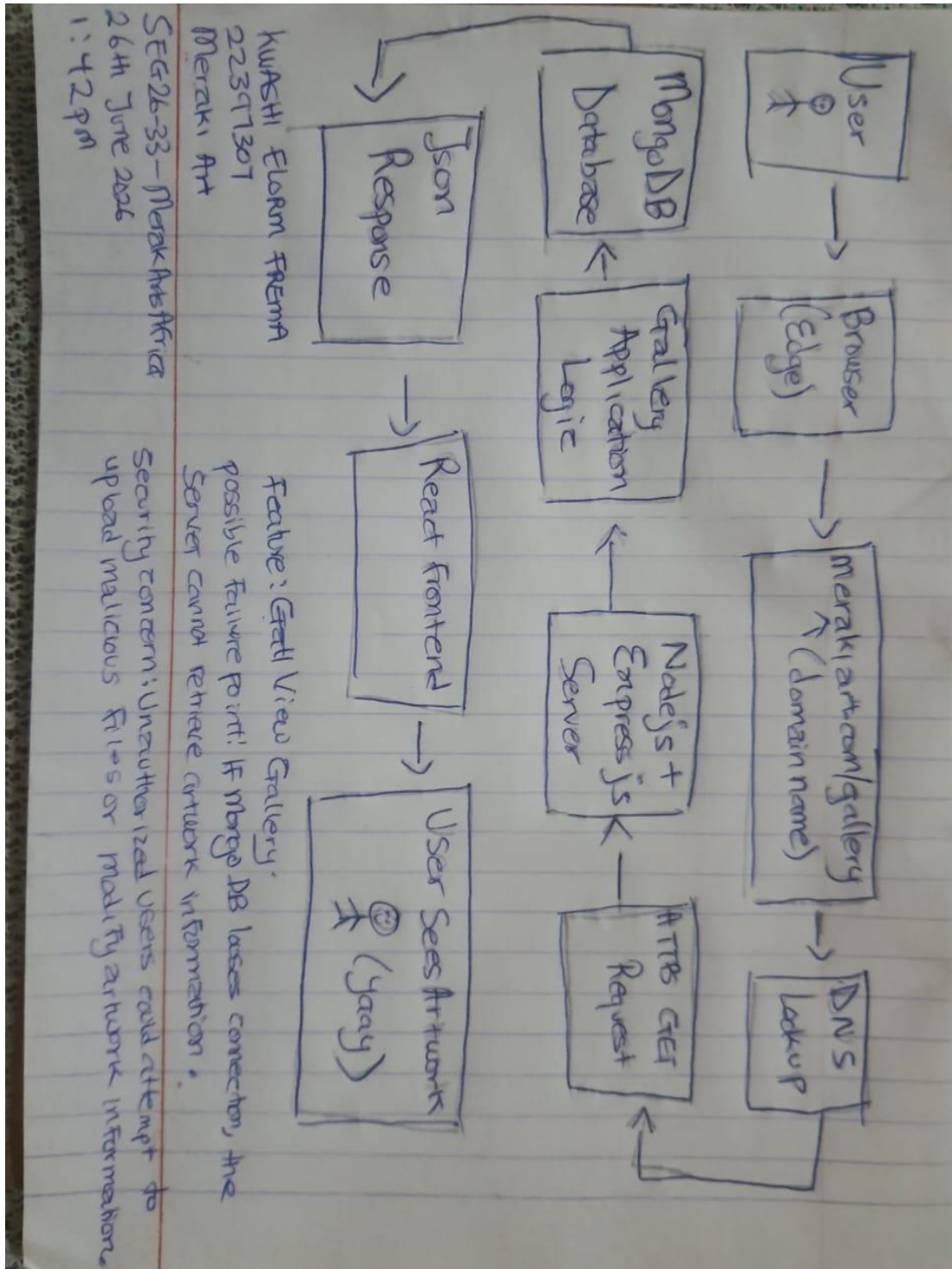
Date: 26th June 2026

AI Disclosure Statement:

I used AI assistance for structuring and formatting the submission page and clarifying submission requirements. The final work was reviewed, verified, and adapted by me. I understand I must be able to explain and defend every part of this submission.

TITLE: Weekly Individual Assignment

4A.



4B. 1. What device is the user on? = The user uses a desktop computer, laptop, tablet, or smartphone to access Meraki Art. The gallery feature will adjust to the screen size so that the user can see artworks on any screen.

2. Is it a browser or mobile app? = Access the system using the web browser. The creation of gallery UI is developed using a react library which executes right in the browser. Accordingly, no native mobile or desktop application is required.

3. What URL, route, screen or endpoint is involved? = A user usually visits the gallery using a URL like <https://merakiart.com/gallery>. After that, the React frontend will communicate with the backend API endpoint, for instance, GET /api/gallery to get the artwork data needed to display the page.

4. What protocol is used? = The app employs HTTPS as its communication protocol. To enhance the security of a website, it is recommended to enable SSL certification for the site's domain.

5. What data is sent? = When you open the gallery page, the browser will send a GET request to retrieve the artwork information. The request can contain optional query parameters including page number, category, artist name, or search terms to filter the results or organize it.

6. What validates the request? = An Express.js server validates incoming requests. Before processing a request, the server checks for the existence of the route, verifies parameter formats, and makes sure permissions or authentication rules are compliant.

7. What database table, collection or external service is touched? = The request works in conjunction with a MongoDB database that stores artwork documents. Most documents contain fields such as title, artist name, image URL, description, category, upload date, which are responsible for loading up the gallery.

8. What response comes back? = The server gives you a JSON object with the data on the art. This response is sent to the React frontend and renders the gallery with images and a modal about related artwork.

9. What happens if validation fails? = If the validation were to fail, then the server is bound to respond back with a 400 Bad Request then a 404 Not Found. The front end will display an error message indicating that the request could not be completed.

10. What happens if the database is down? = When the database is down, the server won't be able to access the information. In this instance, the API returns an error response and the frontend informs that artworks are currently unavailable.

11. What happens if 500 users do this at once? = When a substantial number of users (500 or more) attempts to access the system simultaneously, it creates a high load on the server. To manage this, caching, database indexing, load balancing, and better API design are used to keep it running well without crashing or slowing down.

12. What security or privacy concern exists? = A significant concern with regard to security and privacy is the risk of unauthorized file uploads and changes to artwork data. In response to this, strong input validation, authentication, authorization, and file-handling practices must be implemented in order to safeguard the system and its users.

4C. Oral explanation video link : <https://youtu.be/VJeIY4WzOwY>